Republic of Tunisia
Ministry of High Education and Scientific Research
***
University of Monastir
High Institute of Computer Science and Mathematics of Monastir
Department of Technology
***

Graduation Project

In order to obtain the
National Diploma of Engineering in Applied Sciences and Technology

Major:
Electronics: Microelectronics

Presented by:

Najd Elaoud and Selim Triki

Design and development of the IOBOX-BWS industrial platform for data acquisition, processing, and control

---

Presented *X* **July** 2026 **in front of the jury members :**

Mrs. UNKOWN	President
Mr. UNKOWN	Lecturer
Mr. Abdessalem Ben Abdelali	Academic Supervisor
Mr. Jamel Hmidi	Industrial Supervisor

Academic year: 2025/2026

ACKNOWLEDGEMENTS

I am humbled and grateful to all who have helped me translate these ideas into something concrete that goes far beyond a simple level.

I would like to express my deep gratitude and my sincere thanks to **Mr. Abdessalem BEN ABDELALI** , my Academical supervisor, for accepting being part of my graduation event, for giving me the opportunity to work on this project under her guidance and for her pieces of advice and kind words that have always kept me on my toes.

I would like also to express my deep and sincere gratitude to my industrial supervisor **Mr. jamel HMIDI** for the time he devoted throughout this project and for the valuable information he gave me with interest and understanding.

Last but not least, I am deeply grateful to all the jury members **Mr. UNKOWN** and **Mrs. UNKOWN** for agreeing to evaluate this work.

Without your support, this accomplishment would not have been achieved. I really appreciate it.

DEDICATION

To my precious family, I would never be where I am now without you.

To my dear mother ***Soufia****: You have done more than a mother can do to ensure that her children follow the right path in their lives and studies. I dedicate this work to you as a testimony of my deep love. May God, the Almighty, preserve you and grant you health, long life and happiness.*

To my dear sister ***Nibras****: Your kindness, your precious support, your encouragement throughout my years of study, your love and affection, have been for me the example of perseverance. I find in you the advice of the sister and the support of the friend. No dedication can express my love and gratitude for having you as a sister.*

To my dear Friends, Words can't express how grateful I am to have your friendship in my life. Thank you for supporting me, accepting me, and being wonderful friends.

Lastly, to all the kind people I met during my six years here, to all the institute students, professors, administrators and workers, thank you for being part of my exciting university career.

Najd Elaoud

ABSTRACT

The development of flexible and interoperable industrial platforms is essential for addressing the growing integration challenges of Industrial Internet of Things (IIoT) systems. This report presents the design and development of the IO-Box, a configurable industrial Input/Output platform intended to simplify the acquisition, processing, control, and exchange of data between sensors, actuators, and supervisory systems.

The developed platform is based on an STM32 microcontroller and integrates multiple industrial communication interfaces, peripheral drivers, and sensor management modules within a modular software architecture. The implementation emphasizes hardware abstraction, software reusability, and scalability, enabling efficient integration of heterogeneous devices while reducing development effort.

The resulting solution provides a versatile and reusable platform for industrial monitoring, automation, and IoT applications. By improving interoperability and simplifying system integration, the IO-Box contributes to the development of more efficient and scalable industrial solutions.

Keywords: Industrial IoT, Embedded Systems, IO-Box, Data Acquisition, Industrial Communication Protocols, Sensors, Automation, Software Architecture.

RESUME

Le développement de plateformes industrielles flexibles et interopérables est devenu essentiel pour répondre aux défis croissants d'intégration des systèmes de l'Internet Industriel des Objets (IIoT). Ce rapport présente la conception et le développement de l'IO-Box, une plateforme industrielle d'Entrées/Sorties configurable destinée à simplifier l'acquisition, le traitement, le contrôle et l'échange de données entre les capteurs, les actionneurs et les systèmes de supervision.

La plateforme développée repose sur un microcontrôleur STM32 et intègre plusieurs interfaces de communication industrielle, pilotes de périphériques et modules de gestion de capteurs au sein d'une architecture logicielle modulaire. L'implémentation met l'accent sur l'abstraction matérielle, la réutilisabilité logicielle et l'évolutivité, permettant une intégration efficace de dispositifs hétérogènes tout en réduisant les efforts de développement.

La solution obtenue constitue une plateforme polyvalente et réutilisable pour les applications de supervision industrielle, d'automatisation et d'IoT. En améliorant l'interopérabilité et en simplifiant l'intégration des systèmes, l'IO-Box contribue au développement de solutions industrielles plus efficaces et évolutives.

Mots-clés : Internet Industriel des Objets (IIoT), Systèmes Embarqués, STM32, IO-Box, Acquisition de Données, Protocoles de Communication Industrielle, Automatisation.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

GLOSSARY OF ACRONYMS

BSP: Bourd Support Package

UART : Universal Asynchronous Receiver-Transmitter

LIN: Local Interconnect Network

USB : Universal Serial Bus

CAN : Controller Area Network

I2C : Inter-Integrated Circuit

SPI : Serial Peripheral Interface

DAC : Digital to Analog Converter

ADC : Analog to Digital Converter

GPIO : General Purpose Input Output

PID : Proportional Integral Derivative

GENERAL INTRODUCTION

Industrial systems are increasingly relying on connected devices capable of acquiring, processing, and exchanging information in real time. In such environments, sensors, actuators, and supervisory systems must interact efficiently through various communication interfaces and protocols. However, integrating these heterogeneous components often requires significant development effort, particularly when flexibility, scalability, and interoperability are required.

To address these challenges, Be Wireless Solutions (BWS) launched the development of the IOBox platform, a configurable industrial Input/Output system designed to serve as a universal interface between field devices and monitoring or control systems. The platform is intended to simplify the integration of sensors and actuators while providing data acquisition, signal processing, control, and communication functionalities within a single solution.

The IOBox supports multiple industrial communication protocols and offers both analog and digital input/output capabilities. Its modular architecture enables adaptation to various industrial applications, ranging from environmental monitoring and energy management to process automation and equipment supervision. By centralizing these functionalities in a single platform, the IOBox reduces deployment complexity and accelerates the development of industrial IoT solutions.

The objective of this end-of-study project is to participate in the design and implementation of the IOBox platform. The work focuses on the development of embedded software components, communication interfaces, peripheral drivers, sensor integration, and system architecture required to ensure reliable operation in industrial environments.

This report, which crowns the work entrusted to us, is structured around three chapters:

Chapter 1 presents the general context of the project, the host company, the problem statement, the project objectives, and the adopted development methodology.

Chapter 2 introduces the general concept of the IOBox platform and describes the hardware components, software tools, communication protocols, sensors, and peripherals used throughout the project.

Chapter 3 details the design and implementation of the platform, including the software architecture, peripheral configuration, driver development, communication management, sensor integration, and system validation.

CHAPTER 1

GENERAL PROJECT CONTEXT

1.1 Introduction

This chapter introduces the general context of the graduation project and the environment in which it was carried out. It begins with a presentation of the host company, Be Wireless Solutions (BWS), including its organization, areas of activity, and principal IoT solutions. The chapter then presents the project specifications by defining the identified problem, the concept of the IO-Box platform, the adopted development methodology, and the project's objectives and expected impact. This overview provides the necessary background for understanding the technical developments presented in the following chapters.

1.2 Academic Institution

The Higher Institute of Computer Science and Mathematics of the University of Monastir (ISIMM) is created by the decree n° 1623 of July 09, 2002, is a scientific, public higher education establishment, placed under the supervision of the Ministry of Higher Education and Scientific Research.

ISIMM offers educational programs in the field of Science and Technology that focus on the following disciplines: Computer Science, Mathematics and Electronics[4].

1.3 Host Company Presentation

1.3.1 Company Overview

Be Wireless Solutions (BWS) is a Tunisian engineering company specialized in IoT systems, embedded electronics, and industrial monitoring platforms. The company develops end-to-end solutions combining embedded firmware, hardware design, and cloud-based supervision tools. Its engineering activities cover sensor integration, real-time data acquisition, and communication systems for industrial environments.[1].

The company designs and manufactures connected devices, communication gateways, smart sensors, and cloud-based platforms that enable real-time monitoring, control, and optimization of industrial processes and infrastructures. By combining hardware development, embedded software engineering, wireless communication technologies, and cloud services, BWS provides complete end-to-end IoT solutions adapted to various industrial sectors.

Since its creation, BWS has continuously expanded its activities and established collaborations both nationally and internationally. The company has developed expertise in industrial automation, energy management, smart agriculture, fleet management, environmental monitoring, and remote equipment supervision.

Figure 1.1: Be Wireless Solutions (BWS) logo

1.3.2 Organization Chart

The organizational structure of BWS is composed of several departments that collaborate throughout the development lifecycle of industrial and IoT solutions. These departments include management, research and development, hardware design, embedded software development, cloud services, technical support, and business development.

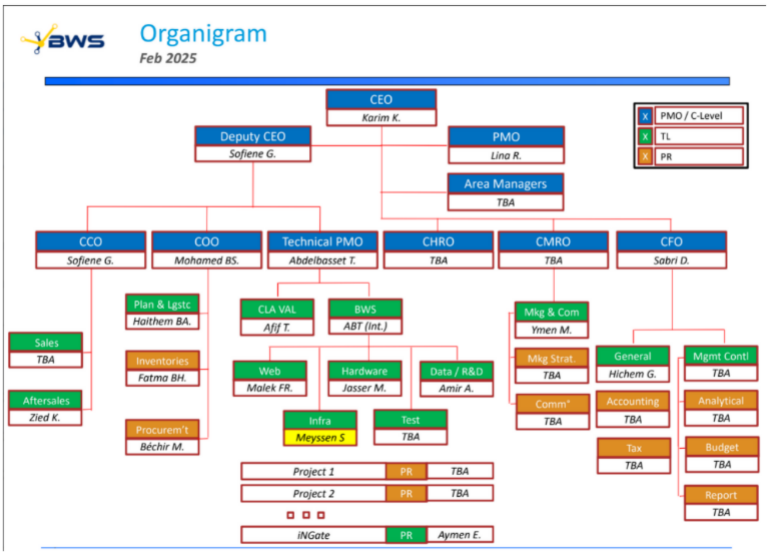

Figure 1.2: Organization chart of Be Wireless Solutions

1.3.3 Main Activities and Services

BWS develops a wide range of industrial IoT solutions designed to improve operational efficiency, resource management, and process supervision.

Energy Monitoring and Management

BWS provides intelligent energy monitoring systems capable of collecting, analyzing, and visualizing electricity consumption data in real time. These solutions help organizations identify energy-intensive equipment, optimize consumption, reduce operational costs, and improve environmental performance.

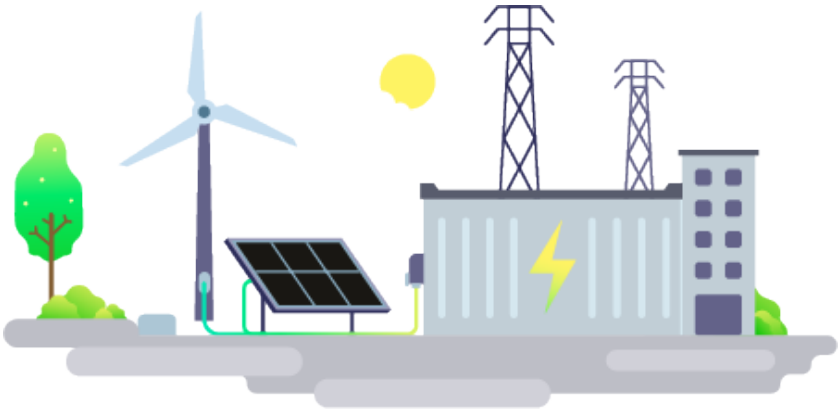

Figure 1.3: Electricity consumption monitoring solution

Water Monitoring Solutions

The company offers smart water management solutions that enable remote meter reading, leak detection, abnormal consumption monitoring, and resource optimization.

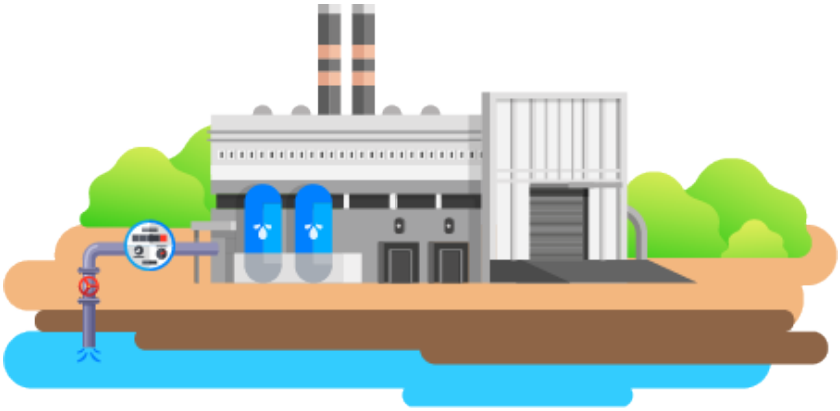

Figure 1.4: Water consumption monitoring solution

Fleet and Fuel Management

BWS develops fleet management systems that provide real-time vehicle tracking, fuel consumption analysis, route optimization, and driver behavior monitoring.

Figure 1.5: Fleet and fuel management solution

Remote Monitoring and Equipment Control

The company provides remote supervision solutions that allow operators to monitor industrial equipment, detect anomalies, generate alerts, and perform remote control operations through dedicated software platforms.

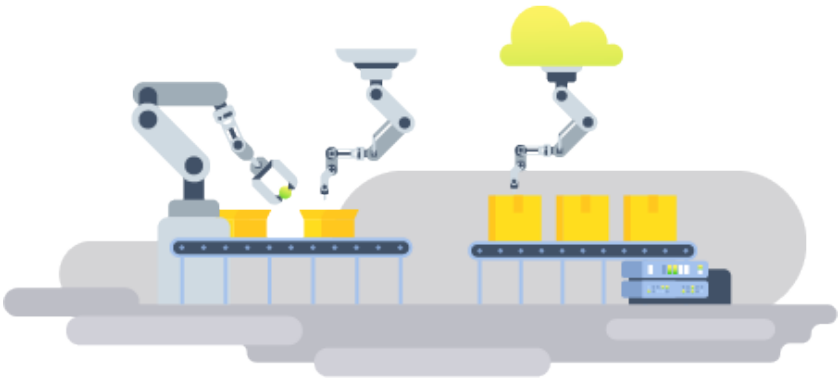

Figure 1.6: Remote equipment monitoring and control

Cold Chain Monitoring

BWS also offers cold-chain monitoring systems intended for warehouses, refrigerated transportation, and pharmaceutical storage facilities. These solutions ensure compliance with temperature requirements and provide instant alerts when abnormal conditions occur.

Figure 1.7: Cold-chain monitoring solution

1.4 Project Specification

1.4.1 Problem Statement

Industrial IoT systems generally integrate a large number of sensors, actuators, and communication interfaces. In many cases, each new application requires dedicated firmware modifications, peripheral configuration, and hardware adaptations. This approach increases development time, reduces software reusability, and complicates system

maintenance.

Moreover, industrial environments often involve heterogeneous communication protocols and interface standards. Managing these technologies through application-specific implementations creates additional complexity and limits system scalability.

To address these limitations, Be Wireless Solutions initiated the development of the IO-Box platform. The objective is to provide a configurable and reusable industrial acquisition and control system capable of simplifying peripheral integration while maintaining flexibility, portability, and maintainability.

1.4.2 Presentation of the IO-Box Platform

The IO-Box is a configurable industrial acquisition and control platform developed to facilitate the integration of sensors, actuators, and communication interfaces within industrial systems.

The platform is built around a modular embedded architecture that separates hardware-dependent components from application-level services. This approach relies on dedicated drivers, a Board Support Package (BSP), and Hardware Abstraction Layer (HAL) mechanisms to ensure portability and maintainability.

The IO-Box supports multiple categories of industrial peripherals, including analog inputs, digital inputs, digital outputs, communication interfaces, and measurement devices. Through a configurable software architecture, the platform can be adapted to different industrial use cases without significant firmware modifications.

By providing a unified interface between hardware resources and application services, the IO-Box reduces integration complexity and accelerates deployment in industrial environments.

1.4.3 Project Objectives

The main objective of this project is the design and development of the IO-Box platform capable of acquiring, processing, regulating, and transmitting industrial data in real time.

More specifically, the project aims to:

- Design and implement a modular industrial acquisition and control platform.
- Provide a unified software abstraction layer for heterogeneous peripherals.
- Support configurable analog and digital input/output functionalities.

- Facilitate communication with external systems through industrial and IoT communication interfaces.
- Develop a modular software architecture based on abstraction layers and reusable software components.
- Ensure platform scalability for future industrial applications and system extensions.

1.4.4 Development Methodology

The development of the IO-Box platform follows the V-Model (Verification and Validation Model), a structured development methodology widely used in embedded and industrial systems.

The V-Model establishes a direct relationship between each development phase and its corresponding verification activity. This approach ensures complete traceability between system requirements, design decisions, implementation activities, and validation tests.

The descending branch of the model includes requirement analysis, system design, and software design activities. The implementation phase constitutes the central stage of the development cycle.

The ascending branch focuses on verification and validation through unit testing, integration testing, system testing, and final acceptance testing.

The adoption of the V-Model contributes to improving software quality, reducing development risks, and facilitating project management throughout the entire lifecycle.

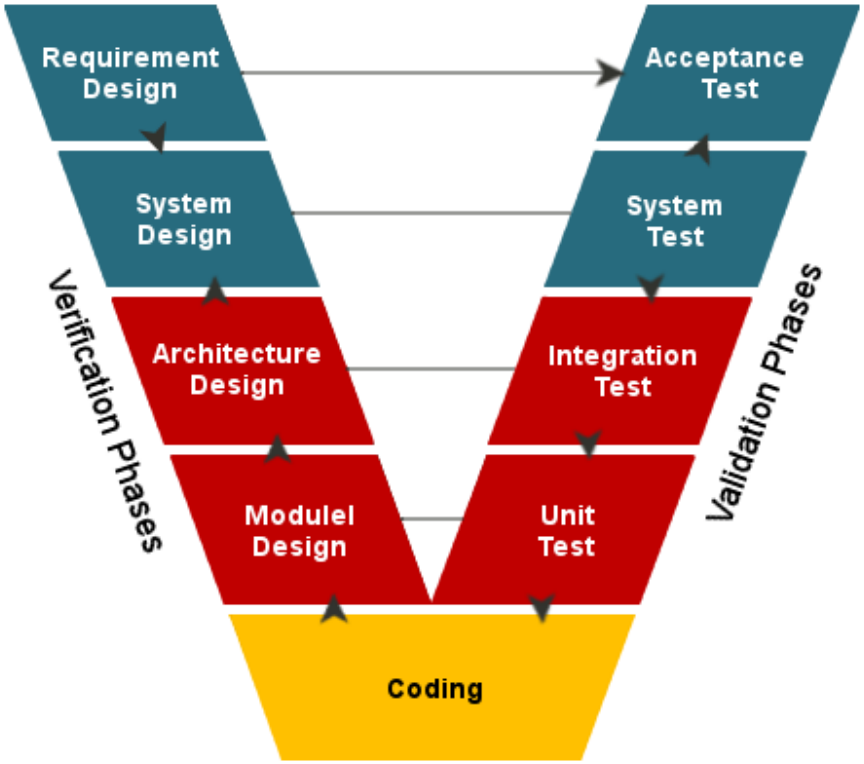

Figure 1.8: V-Model development lifecycle

1.4.5 Expected Contributions

The developed IO-Box platform is expected to significantly simplify the integration of industrial sensors and actuators while reducing deployment and maintenance efforts.

By providing a unified hardware and software architecture, the platform improves interoperability between devices and facilitates future system extensions. Furthermore, its modular design promotes software reuse and accelerates the development of new industrial applications.

The developed platform contributes to reducing development effort and deployment time by providing a standardized and reusable embedded architecture capable of supporting a wide range of industrial applications.

1.5 Conclusion

This chapter introduced the general framework of the project. The academic institution and the host company were first presented, followed by an overview of the company's activities and technological solutions. The project specifications, problem statement, objectives, and development methodology were then detailed.

The next chapter focuses on the requirements analysis and system study, providing a detailed description of the functional and technical requirements that guided the design and implementation of the IO-Box platform.

CHAPTER 2

GENERAL CONCEPT AND SYSTEM COMPONENTS

2.1 Introduction

This chapter presents the technical environment and the main components used in the development of the IO-Box platform. It introduces the hardware and software resources that form the basis of the system, including the STM32G474 microcontroller, external peripherals, sensors, development tools, and communication protocols.

The chapter also describes the principal STM32 peripherals employed within the project, such as Analog-to-Digital Converters (ADC), Digital-to-Analog Converters (DAC), timers, and Pulse Width Modulation (PWM) modules. Understanding these technologies is essential for comprehending the design and implementation choices that will be detailed in the following chapters.

2.2 General Concept of the IO-Box Platform

The IO-Box is a configurable industrial Input/Output platform designed to facilitate the integration of sensors, actuators, and industrial communication interfaces within IoT and automation systems.

The platform acts as an intermediary between field devices and supervisory systems by acquiring data from sensors, processing the collected information, controlling external devices, and exchanging information through various communication protocols.

Its modular architecture allows different hardware modules and software functionalities to be integrated according to application requirements. This flexibility makes

the IO-Box suitable for a wide range of industrial monitoring, automation, and data acquisition applications.

The platform combines high-performance embedded processing, multiple communication interfaces, analog and digital signal acquisition capabilities, and non-volatile storage mechanisms in a single integrated solution.

2.3 Hardware components

2.3.1 STM32G474MBTx Microcontroller

At the heart of the IO-Box hardware architecture lies the STM32G474 microcontroller, a high-performance device from STMicroelectronics. It is based on the ARM Cortex-M4 core with floating-point unit (FPU), making it particularly well-suited for real-time signal processing and control applications.

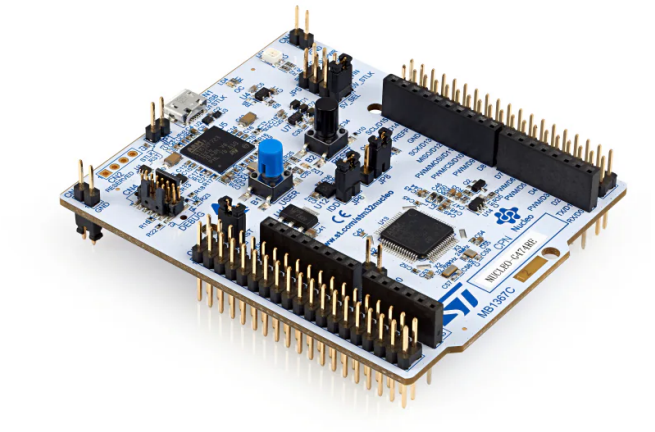

Figure 2.1: STM32G474 Microcontroller (physical package)

The STM32G474 was selected for several reasons:

- **Processing Capabilities**: Equipped with a 170 MHz CPU and DSP instructions, it enables efficient execution of filtering, regulation, and control algorithms.
- **Rich Peripheral Set**: Provides multiple ADC channels, DAC outputs, PWM timers, and communication interfaces (UART, SPI, I^2C, CAN FD, LIN, USB), ensuring compatibility with diverse sensors and actuators.

- **Memory Resources**: Integrates up to 128 KB SRAM and 512 KB Flash, allowing the implementation of complex firmware while maintaining flexibility.
- **Analog Performance**: Features high-resolution ADCs and DACs, supporting precise acquisition and generation of industrial signals.
- **Scalability and Ecosystem**: Benefits from the STM32Cube ecosystem, including HAL libraries, BSP drivers, and development tools that simplify firmware design and portability.

Table 2.1: Product Attributes — STM32G474 Microcontroller

Category	Integrated Circuits, Embedded Systems, Microcontrollers
Manufacturer	STMicroelectronics
Series	STM32G4
Processor Core	ARM Cortex-M4F
Core Size	32-Bit
Speed	170 MHz
Number of I/O	52
Program Memory Size	512 KB Flash
RAM Size	128 KB
Data Converters	ADC: 26 × 12-bit; DAC: 7 × 12-bit
Supply Voltage	1.71 V – 3.6 V
Oscillator Type	Internal
Operating Temperature	–40 °C to +125 °C
Mounting Type	Surface Mount
Package	64-LQFP (10 × 10 mm)
Base Product Number	STM32G474

Within the IO-Box, the STM32G474 acts as the central processing unit, coordinating signal acquisition, local data processing, and actuator control. It also supervises communication with external systems through multiple industrial protocols, making it a key enabler of the project's objectives.

2.3.2 EEPROM Memory: M95M01-RDW6TP

To complement the STM32 microcontroller and ensure reliable data storage, the IO-Box integrates an external EEPROM memory, specifically the M95M01-RMN6TP from STMicroelectronics. This device provides non-volatile storage for configuration parameters, calibration data, and system logs, allowing the platform to retain critical information even after power cycles.

The M95M01-RMN6TP is a 1-Mbit serial EEPROM organized as 131,072 × 8 bits. It features a fast access time of 25 ns and communicates via the SPI bus, ensuring efficient integration with the STM32G474. Packaged in an 8-pin SOIC format, it offers a compact solution suitable for embedded applications. Its endurance of up to one million write cycles and data retention of 40 years make it a robust choice for industrial environments where reliability is essential.

Table 2.2: Key Characteristics of M95M01-RDW6TP EEPROM

Feature	**Specification**
Manufacturer	STMicroelectronics
Type	Serial EEPROM
Capacity	1 Mbit (131,072 × 8 bits)
Interface	SPI
Access Time	25 ns
Package	SOIC-8
Endurance	1,000,000 write cycles
Data Retention	40 years
Operating Voltage	1.8 V – 5.5 V

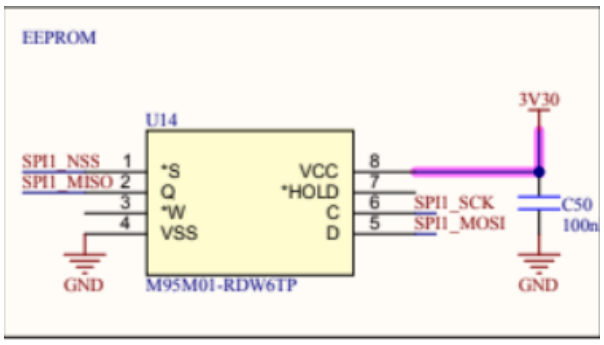

Figure 2.2: M95M01-RDW6TP EEPROM memory chip

2.3.3 Sensors

In the IOBox project, several sensors were integrated to enable environmental and physical measurements. Our work focused not only on interfacing these sensors with the STM32G474 microcontroller, but also on configuring their registers, managing communication protocols, and performing calibration to ensure accurate data acquisition. Each sensor was studied in detail, and its characteristics were documented to highlight its role within the system.

ENS210 Sensor

The ENS210 is a digital sensor from ScioSense that integrates both temperature and humidity measurement in a compact package. It communicates via an I^2C interface, making it easy to integrate with microcontrollers such as the STM32G474. Its high accuracy and low power consumption make it suitable for environmental monitoring in embedded systems.

Table 2.3: Key Characteristics of ENS210 Sensor

Feature	**Specification**
Manufacturer	ScioSense
Type	Temperature + Humidity Sensor
Interface	I^2C (16-bit output registers)
Supply Voltage	1.71 V – 3.6 V
Temperature Range	–40 °C to +100 °C
Temperature Accuracy	±0.15 °C
Humidity Range	0 – 100 % RH
Humidity Accuracy	±2 % RH
Package	4-QFN (2 × 2 mm)

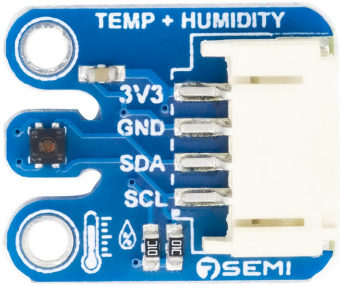

Figure 2.3: ENS210 digital temperature and humidity sensor

2.3.4 OPT3001DNPR Sensor

The OPT3001DNPR is a digital ambient light sensor from Texas Instruments. It is designed to measure the intensity of visible light and provide results in lux units, making it ideal for applications such as automatic brightness control, display backlight adjustment, and environmental monitoring. The sensor communicates via the I^2C protocol and features built-in calibration and linear response across a wide dynamic range.

In the IO-Box project, the OPT3001DNPR was integrated to provide accurate light measurements. Work was carried out on its internal registers to configure measure-

ment modes, thresholds, and calibration routines, ensuring reliable data acquisition for real-time applications.

Table 2.4: Key Characteristics of OPT3001DNPR Sensor

Feature	**Specification**
Manufacturer	Texas Instruments
Type	Ambient Light Sensor
Interface	I^2C
Supply Voltage	1.6 V – 3.6 V
Measurement Range	0.01 lux – 83,000 lux
Output Format	16-bit digital result in lux
Accuracy	±10% typical
Operating Temperature	–40 °C to +85 °C
Package	6-Pin WSON (2 × 2 mm)

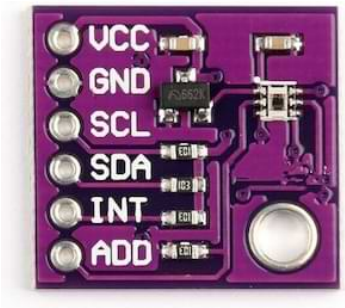

Figure 2.4: OPT3001DNPR ambient light sensor

HTS221TR Sensor

The HTS221TR is a capacitive digital humidity and temperature sensor developed by STMicroelectronics. It integrates sensing elements and signal conditioning circuitry within a compact package, providing calibrated measurements through a standard I^2C interface. The sensor is designed for low-power applications while maintaining good measurement accuracy, making it suitable for environmental monitoring and industrial IoT systems.

Within the IO-Box platform, the HTS221TR is used to acquire ambient temperature and relative humidity data. Its factory-calibrated coefficients simplify integration and improve measurement reliability. Communication with the STM32G474 microcontroller is achieved through the I^2C bus, and dedicated software drivers were developed to configure the device and process sensor data.

Table 2.5: Key Characteristics of HTS221TR Sensor

Feature	**Specification**
Manufacturer	STMicroelectronics
Type	Temperature + Humidity Sensor
Interface	I^2C / SPI
Supply Voltage	1.7 V – 3.6 V
Temperature Range	-40 °C to +120 °C
Temperature Accuracy	±0.5 °C (typ.)
Humidity Range	0 – 100 % RH
Humidity Accuracy	±3.5 % RH (typ.)
Output Resolution	16-bit
Package	HLGA-6 (2 × 2 mm)

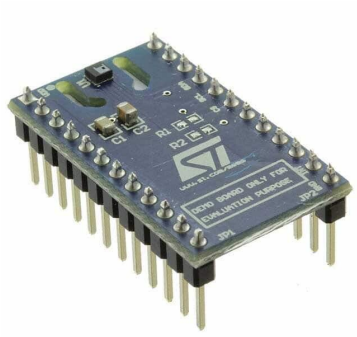

Figure 2.5: HTS221TR temperature and humidity sensor

2.4 Software tools

2.4.1 Visual Studio Code

Visual Studio Code (VSC) is a lightweight yet powerful source code editor optimized for modern software development. It supports multiple programming languages and provides advanced features such as syntax highlighting, intelligent code completion, and debugging tools. In this project, Visual Studio Code was primarily used for version control and collaboration through Git. Extensions such as Git Graph enabled efficient visualization of commit history, branch management, and tracking of project evolution. Visual Studio Code is free and available on all major platforms (Linux, macOS, and Windows)[2].

Figure 2.6: Visual Studio Code interface

2.4.2 IAR Embedded Workbench

IAR Embedded Workbench (IAR EW) is a professional integrated development environment (IDE) dedicated to embedded systems programming, particularly for microcontrollers such as the STM32 family. It provides a highly optimized C/C++ compiler, advanced debugging capabilities, and efficient project management tools. In this project, IAR EW was used for firmware development, compilation, and debugging. Its powerful optimization features contributed to generating efficient and reliable embedded code, which is critical for real-time applications and resource-constrained systems[3].

Figure 2.7: IAR Embedded Workbench IDE

2.4.3 STM32CubeMX

STM32CubeMX is a graphical configuration tool developed by STMicroelectronics for STM32 microcontrollers. It allows developers to easily configure peripherals, clock settings, and pin assignments through an intuitive interface. The tool also generates initialization code based on the selected configuration, significantly reducing development time and minimizing configuration errors. In this project, STM32CubeMX was used to set up the microcontroller hardware abstraction layer (HAL), initialize peripherals such as GPIO, UART, ADC, and timers, and generate the base project structure later integrated into the development environment[5].

Figure 2.8: STM32CubeMX configuration interface

2.4.4 GitLab

GitLab is employed as a collaborative platform for version control and project management. It enables developers to share code, track changes, and maintain repositories securely. Through GitLab, the team ensures transparency, reproducibility, and efficient collaboration, which are critical for academic and industrial projects alike.

Figure 2.9: GitLab logo

2.5 Communication Protocols

2.5.1 I2C Protocol

Inter-Integrated Circuit (I2C) is a synchronous, half-duplex serial communication protocol widely used for communication between integrated circuits over short distances. It relies on two bidirectional lines: the Serial Data Line (SDA) and the Serial Clock Line (SCL), both requiring pull-up resistors. The protocol follows a master-slave architecture, where the master initiates communication and generates the clock signal.

Each device connected to the bus is identified by a unique address. Communication is initiated by a START condition, followed by the transmission of the slave address and a read/write bit. Data is then exchanged in 8-bit frames, each followed by an acknowledgment (ACK/NACK) bit. The communication is terminated by a STOP condition. I2C supports different speed modes, including Standard Mode (100 kHz), Fast Mode (400 kHz), and higher-speed variants.

The I2C frame structure is illustrated in Figure 2.10.

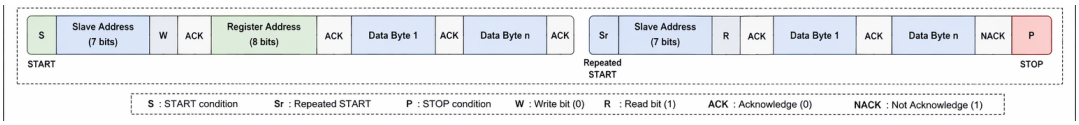

Figure 2.10: I2C frame format

In this project, the I2C interface of the STM32G474MBTx is configured using the HAL library generated by STM32CubeMX. It is used to communicate with external peripherals such as sensors and low-speed devices. The implementation relies on interrupt or polling-based data transfers depending on timing constraints. Particular attention is given to bus stability, including proper pull-up resistor sizing and handling of potential communication errors such as bus busy states or missing acknowledgments.

As shown in Figure 2.11, the I2C bus allows multiple slave devices.

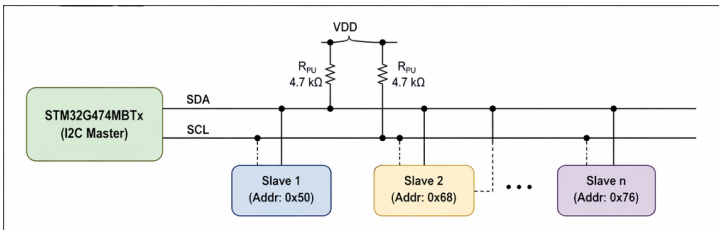

Figure 2.11: I2C communication bus architecture

2.5.2 Serial Peripheral Interface (SPI)

The Serial Peripheral Interface (SPI) is a synchronous serial communication protocol widely used in embedded systems. It enables high-speed data exchange between a microcontroller and peripheral devices such as sensors, memory modules, or communication transceivers. SPI operates in a master–slave configuration, where the microcontroller typically acts as the master, controlling the clock and data flow.

In the IO-Box project, SPI is employed to ensure reliable and efficient communication with external modules that require fast data transfer. Its simplicity and speed make it suitable for real-time applications, particularly when multiple peripherals must be interfaced with the STM32 microcontroller.

SPI communication is based on a clock signal generated by the master device. Each bit of data is shifted out on the MOSI line while simultaneously another bit is shifted in on the MISO line. This process is synchronized by the clock (SCLK), ensuring deterministic timing. The communication sequence typically follows these steps: the master activates the Chip Select (CS) line of the desired slave, provides the clock signal, transmits data bit-by-bit on MOSI while receiving data on MISO, samples the data according to the configured mode, and finally deactivates the CS line to end the session. This mechanism allows full-duplex communication, meaning data can be transmitted and received simultaneously.

SPI supports four modes of operation (Mode 0 to Mode 3), determined by the configuration of clock polarity (CPOL) and clock phase (CPHA). This flexibility ensures compatibility with a wide range of peripheral devices. Key characteristics of SPI include its high speed (operating at several MHz), simple wiring with four main signals (MOSI, MISO, SCLK, CS), and scalability through multiple slave connections.

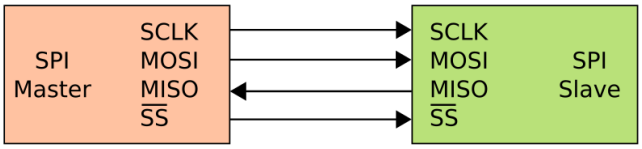

Figure 2.12: Basic SPI communication between master and slave devices

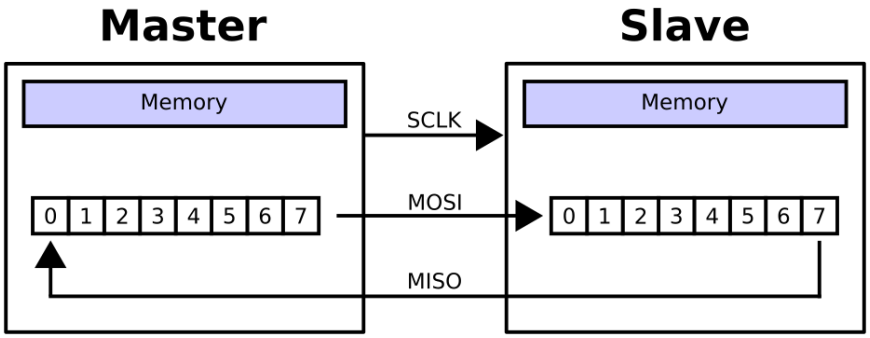

Figure 2.13: Comparison of Big-endian and Little-endian byte ordering in memory

2.5.3 UART Protocol

Universal Asynchronous Receiver-Transmitter (UART) is an asynchronous serial communication protocol that enables full-duplex data exchange between devices without a shared clock signal. Data is transmitted over two lines: Transmit (TX) and Receive (RX). Synchronization between devices is achieved by configuring identical communication parameters, including baud rate, data bits, parity, and stop bits.

Each data frame typically consists of a start bit, followed by a configurable number of data bits (usually 8), an optional parity bit for error detection, and one or more stop bits. Unlike synchronous protocols, UART does not include a clock line, making it simpler but more sensitive to timing mismatches.

In this project, UART is used for communication between the STM32 microcontroller and external systems such as a host computer or other embedded modules. It plays a key role in debugging, logging, and real-time data exchange. The implementation uses the STM32 HAL drivers with support for interrupt-driven and DMA-based transmission to ensure efficient and non-blocking communication. Error handling mechanisms, including overrun, framing, and noise errors, are also considered to improve communication reliability.

The UART frame structure is illustrated in Figure 2.14.

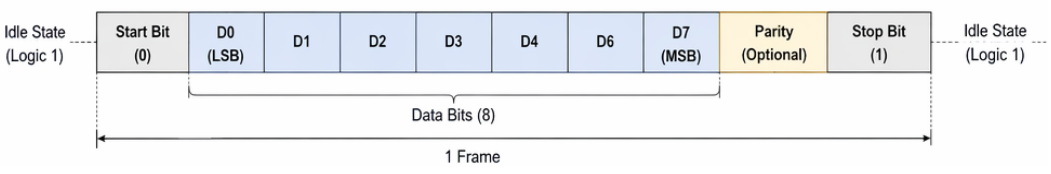

Figure 2.14: UART frame format

2.5.4 RS485 Communication

RS485 is a robust differential serial communication standard widely used in industrial environments due to its high noise immunity and long-distance capabilities. Unlike single-ended UART communication, RS485 transmits data over a differential pair of lines (A and B), where the information is encoded as a voltage difference between the two conductors. This approach significantly reduces susceptibility to electromagnetic interference and ground potential differences.

The RS485 standard supports half-duplex communication in multi-drop configurations, allowing multiple nodes to share the same bus. However, only one device can transmit at a time, which requires proper bus arbitration at the software level or through protocol design.

In this project, RS485 communication is implemented using the UART peripheral of the STM32G474MBTx combined with an external RS485 transceiver. The transceiver converts TTL-level UART signals into differential signals suitable for the bus. A dedicated GPIO pin is used to control the Driver Enable (DE) and Receiver Enable (RE) pins of the transceiver, allowing the system to switch between transmission and reception modes.

Before transmission, the microcontroller sets the DE pin high to enable the driver stage. After sending the data frame, the DE pin is reset to return the transceiver to reception mode. This timing control is critical to avoid bus contention. The communication is typically used for reliable data exchange in noisy environments, especially when longer cable lengths are required compared to standard UART.

Figure 2.15 illustrates a typical RS485 bus configuration.

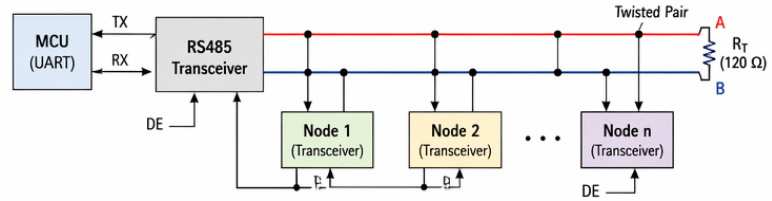

Figure 2.15: RS485 differential bus with multi-node configuration

2.5.5 Modbus Protocol

Modbus is one of the most widely used industrial communication protocols for exchanging data between electronic devices. Originally developed by Modicon in 1979, it follows a client-server (master-slave) architecture and enables communication between

programmable logic controllers (PLCs), sensors, actuators, Human-Machine Interfaces (HMIs), and embedded systems.

Modbus defines a standardized message structure that allows devices from different manufacturers to communicate using a common protocol. Two main variants are commonly used: Modbus RTU, which operates over serial communication interfaces such as RS232 and RS485, and Modbus TCP, which operates over Ethernet networks.

In the IO-Box project, Modbus RTU is implemented over the RS485 physical layer to ensure reliable communication in industrial environments. The STM32G474 microcontroller acts as either a Modbus master or slave depending on the application requirements. Through Modbus registers, external systems can access sensor measurements, configuration parameters, status information, and control commands.

A Modbus RTU frame consists of four main fields:

- **Slave Address**: Identifies the target device.
- **Function Code**: Specifies the requested operation.
- **Data Field**: Contains transmitted data or register information.
- **CRC Field**: Provides error detection for frame integrity.

The protocol offers several advantages for industrial applications:

- Simplicity of implementation.
- Wide industrial adoption.
- Interoperability between devices from different vendors.
- Reliable communication over long distances using RS485.
- Easy integration with supervisory and SCADA systems.

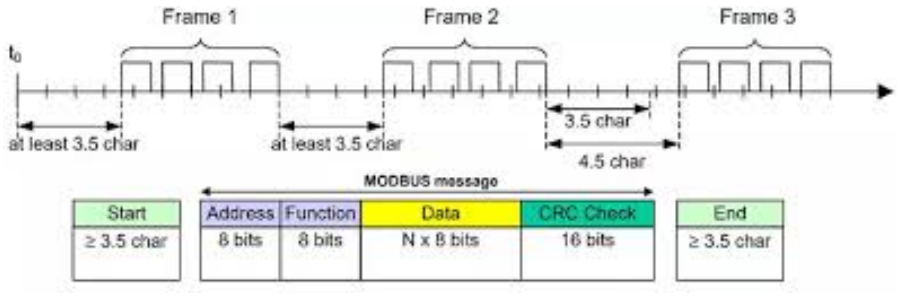

Figure 2.16: Typical Modbus RTU frame structure

2.5.6 Controller Area Network (CAN)

The Controller Area Network (CAN) is a robust serial communication protocol originally developed for automotive applications, but now widely used in industrial and embedded systems. It enables reliable data exchange between multiple devices without requiring a central host, making it ideal for distributed control systems.

In the IO-Box project, CAN is employed to ensure communication with industrial equipment and sensors that demand high reliability and fault tolerance. Its ability to handle multiple nodes on a single bus makes it particularly suitable for environments where numerous devices must share information in real time.

CAN communication is based on a message-oriented protocol. Instead of addressing specific devices directly, each message carries an identifier that defines its priority and content. This allows multiple nodes to listen simultaneously and decide whether to act on the received data. The arbitration mechanism ensures that when two nodes attempt to transmit at the same time, the message with the highest priority (lowest identifier value) wins, while the other node waits until the bus is free. This guarantees deterministic communication without collisions.

Key characteristics of CAN include:

- **Multi-master capability**: Any node can initiate communication when the bus is idle.
- **Error detection and handling**: Built-in mechanisms such as CRC checks, acknowledgment bits, and error frames ensure data integrity.
- **Priority-based arbitration**: Messages are prioritized by identifiers, ensuring that critical data is transmitted first.
- **Scalability**: Supports up to 112 nodes on a single bus, making it suitable for complex systems.
- **Speed**: Operates up to 1 Mbps in classical CAN, and higher in CAN FD (Flexible Data-Rate), which extends payload size and transmission speed.

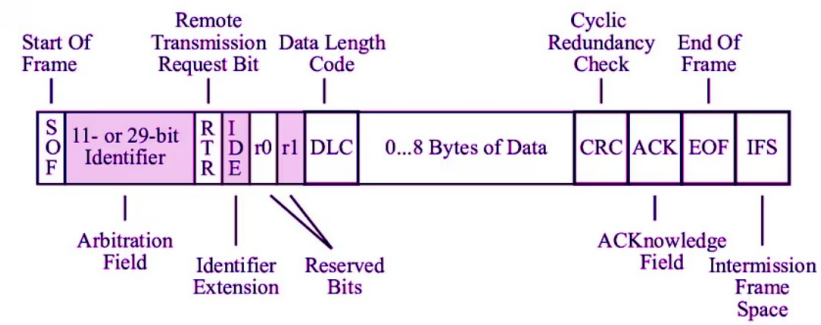

Figure 2.17: Basic CAN frame

In practice, CAN uses two differential lines, CAN_H and CAN_L, to transmit data. This differential signaling improves noise immunity and allows reliable communication even in electrically harsh environments. Each frame consists of fields such as the start of frame, identifier, control bits, data payload, CRC, and end of frame, ensuring structured and error-resistant communication.

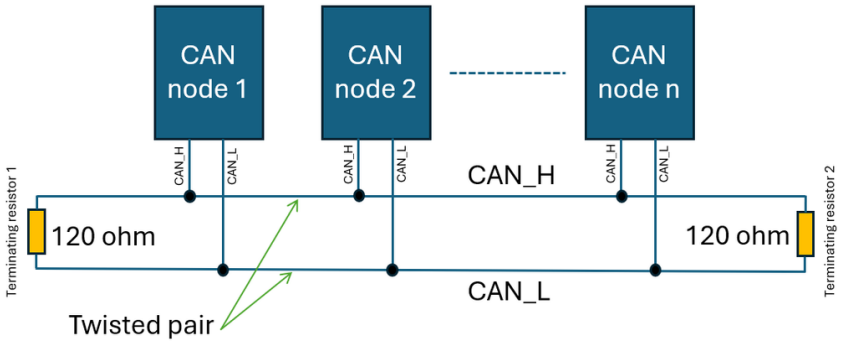

Figure 2.18: Basic CAN bus communication with multiple nodes

2.5.7 LIN Protocol

Local Interconnect Network (LIN) is a low-cost serial communication protocol designed for distributed control systems, particularly in automotive applications. It operates on a single-wire physical layer and follows a strict master-slave architecture where the master node controls the entire bus communication schedule.

Unlike event-driven protocols, LIN is time-triggered, meaning communication is organized into predefined frames sent at fixed intervals. Each frame consists of a header transmitted by the master and a response transmitted by a slave node. The header includes a break field (used to signal the start of a frame), a synchronization byte, and an identifier field that determines which node should respond.

The data response typically contains up to 8 bytes, followed by a checksum used for error detection. LIN achieves deterministic timing by enforcing strict scheduling, making it suitable for non-critical but predictable control tasks.

In this project, LIN communication is implemented using the STM32 UART peripheral configured in LIN mode. The hardware support simplifies break detection and synchronization field handling, reducing software complexity. The system can therefore reliably detect frame boundaries and process incoming data with minimal CPU overhead.

The LIN frame structure is shown in Figure 2.19.

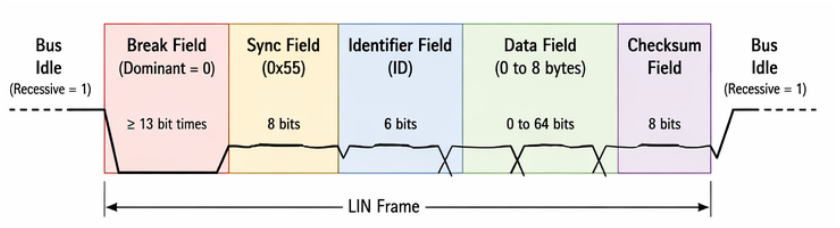

Figure 2.19: LIN frame structure

2.6 Peripherals

2.6.1 Digital-to-Analog Converter (DAC)

The Digital-to-Analog Converter (DAC) is an analog output peripheral integrated within the STM32G474MBTx microcontroller. It converts digital values stored in memory into continuous analog voltage levels with a resolution defined by the DAC bit width. This allows the generation of precise voltage references or analog waveforms such as sine waves, ramps, or arbitrary signals.

The DAC can operate in several modes, including software-triggered mode, timer-triggered mode, and DMA-driven mode. In timer-triggered mode, a hardware timer periodically updates the DAC output, ensuring precise and deterministic signal timing. When combined with Direct Memory Access (DMA), the DAC can continuously output precomputed waveform data without CPU intervention, significantly improving efficiency.

In this project, the DAC is used for controlled analog signal generation required for system-level testing and control purposes. Its integration with timers ensures synchronization with other system events, enabling deterministic behavior in real-time applications. This is particularly useful in embedded control systems where stable analog references or periodic waveforms are required.

An example of DAC signal generation is illustrated in Figure 2.20.

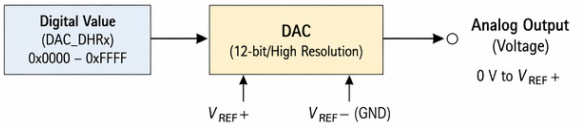

Figure 2.20: Example of analog signal generated by DAC

2.6.2 Analog to Digital Converter (ADC)

The Analog-to-Digital Converter (ADC) is a fundamental peripheral in embedded systems, responsible for transforming continuous analog signals into discrete digital values that can be processed by the microcontroller. This conversion is essential for applications where physical phenomena such as temperature, pressure, or current must be measured and interpreted by digital logic.

In the IOBox project, ADCs are employed to acquire signals from sensors and external modules, enabling precise monitoring and control. The STM32G474 microcontroller integrates multiple high-resolution ADC channels, allowing simultaneous sampling of different inputs. This capability is particularly important in industrial contexts where multiple signals must be captured in real time.

The operation of an ADC follows a well-defined process. The analog input is sampled at regular intervals determined by the sampling frequency. Each sample is then quantized into a digital value according to the converter's resolution (e.g., 12-bit or 16-bit). The resulting digital word represents the amplitude of the analog signal at that instant. The accuracy of this conversion depends on factors such as resolution, sampling rate, and reference voltage stability.

Key characteristics of ADCs include:

- **Resolution**: Defines the number of discrete levels available.
- **Sampling Rate**: Determines how frequently the analog signal is measured, affecting the fidelity of dynamic signals.
- **Reference Voltage**: Sets the maximum measurable input range, ensuring proper scaling of the digital output.
- **Input Channels**: Multiple channels allow acquisition from several sensors simultaneously.
- **Conversion Modes**: Support for single, continuous, and DMA-driven conver-

sions, enabling flexible integration into firmware.

Within the IO-Box, ADCs are used to capture sensor data such as voltage levels, current measurements, or other analog signals. These values are then processed by the firmware to generate meaningful information, trigger control actions, or communicate results to external supervisory systems.

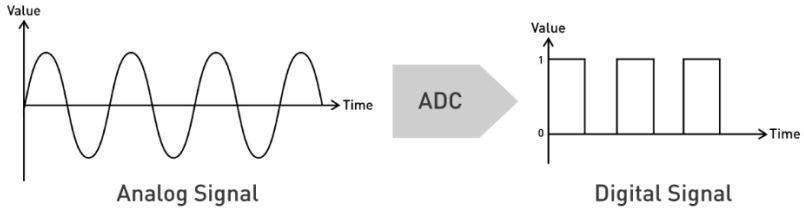

Figure 2.21: Basic ADC conversion from analog input to digital output

2.6.3 Timers

Timers are fundamental peripherals in the STM32G474 microcontroller, designed to provide accurate time base generation, event counting, and signal measurement. They operate by incrementing or decrementing a counter register according to a clock source, and can trigger actions when predefined values are reached.

Key functionalities of timers include:

- **Time Base Generation**: Creating precise delays or periodic events.
- **Input Capture**: Measuring external signal characteristics such as frequency or pulse width.
- **Output Compare**: Generating events when the counter matches a reference value.
- **Synchronization**: Coordinating multiple timers for complex control tasks.
- **Advanced Features**: Dead-time insertion, break inputs, and complementary outputs for safe power electronics control.

Timers are essential for scheduling tasks, measuring sensor signals, and driving peripherals that require precise timing.

2.6.4 Pulse Width Modulation (PWM)

Pulse Width Modulation (PWM) is a technique used to encode analog information into a digital waveform by varying the duty cycle of a periodic signal. In embedded systems, PWM is widely used for motor control, LED dimming, and power regulation.

PWM signals in the STM32G474 are generated using timers. The timer counts up to a predefined value stored in the auto-reload register (ARR), while the compare register (CCR) defines the switching point of the output. The ratio between the high time and the total period represents the duty cycle. By adjusting ARR and CCR, both frequency and duty cycle can be controlled with high precision.

Key characteristics of PWM include:

- **Frequency**: Determined by prescaler and ARR configuration.
- **Duty Cycle**: Defined by CCR relative to ARR.
- **Resolution**: Limited by timer bit width (typically 16-bit).
- **Complementary Outputs**: Enable safe control of MOSFET drivers in half-bridge or full-bridge circuits.
- **Applications**: Motor drivers, LED dimming, switching converters, and DAC emulation.

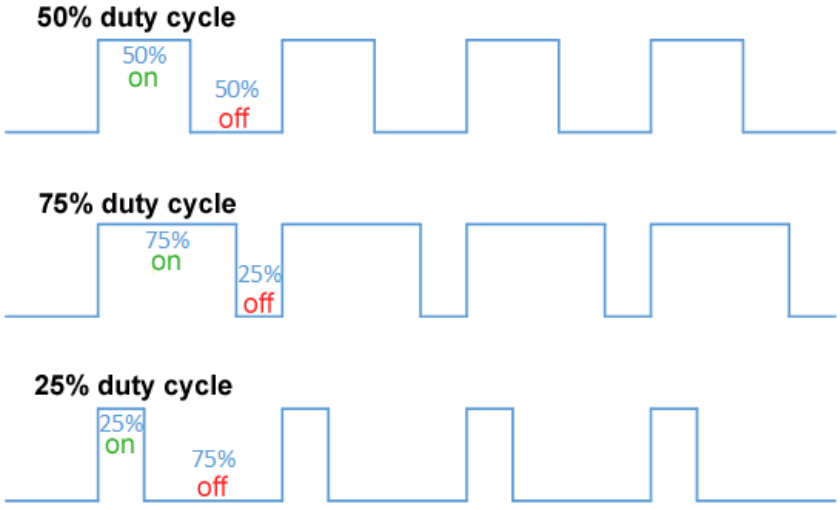

Figure 2.22: Illustration of PWM signals with different duty cycles (25%, 50%, and 75%)

2.6.5 Conclusion

In this chapter, we presented the general concept and system components of the IO-Box project. We began with the synoptic diagram, which outlined the main functional phases of acquisition, filtering, regulation, and transmission/actuation. We then introduced the software architecture, organized into four distinct layers (HAL, BSP, Manager, and Application), and complemented this with the hardware architecture, highlighting the STM32G474 microcontroller, power supply design, communication interfaces, and peripheral modules. Finally, we described the development tools and protocols that support the implementation of the system.

By establishing this technical foundation, Chapter 3 has clarified the resources and methodologies that will be employed in the realization phase. The next chapter will be dedicated to the practical implementation of the IO-Box, detailing the firmware design, register configuration, BSP development, and integration of sensors and actuators into a coherent and functional platform.

CHAPTER 3

DESIGN AND PRACTICAL IMPLEMENTATION

3.1 Introduction

BIBLIOGRAPHY

[1] BWS. consulted on 04/13/2026. https://bewireless-solutions.com/en/home/.

[2] VS code. consulted on 04/13/2026. https://code.visualstudio.com/.

[3] IAR EW. consulted on 04/13/2026. https://www.iar.com/.

[4] ISIMM. consulted on 04/13/2026. http://www.isimm.rnu.tn/public/.

[5] STM32CubeMX. consulted on 04/13/2026. https://www.st.com/en/development-tools/stm32cubemx.html.